

particularly within dynamic or complex environments. This process transforms a high-level objective into a structured plan composed of discrete, executable steps.

In domains such as procedural task automation, planning is used to orchestrate complex workflows. For example, a business process like onboarding a new employee can be decomposed into a directed sequence of sub-tasks, such as creating system accounts, assigning training modules, and coordinating with different departments. The agent generates a plan to execute these steps in a logical order, invoking necessary tools or interacting with various systems to manage dependencies.

Within robotics and autonomous navigation, planning is fundamental for state-space traversal. A system, whether a physical robot or a virtual entity, must generate a path or sequence of actions to transition from an initial state to a goal state. This involves optimizing for metrics such as time or energy consumption while adhering to environmental constraints, like avoiding obstacles or following traffic regulations.

This pattern is also critical for structured information synthesis. When tasked with generating a complex output like a research report, an agent can formulate a plan that includes distinct phases for information gathering, data summarization, content structuring, and iterative refinement. Similarly, in customer support scenarios involving multi-step problem resolution, an agent can create and follow a systematic plan for diagnosis, solution implementation, and escalation.

In essence, the Planning pattern allows an agent to move beyond simple, reactive actions to goal-oriented behavior. It provides the logical framework necessary to solve problems that require a coherent sequence of interdependent operations.

Hands-on code (Crew AI)

The following section will demonstrate an implementation of the Planner pattern using the Crew AI framework. This pattern involves an agent that first formulates a multi-step plan to address a complex query and then executes that plan sequentially.

```
import os
from dotenv import load_dotenv
from crewai import Agent, Task, Crew, Process
from langchain_openai import ChatOpenAI

# Load environment variables from .env file for security
load_dotenv()
```

```

# 1. Explicitly define the language model for clarity
llm = ChatOpenAI(model="gpt-4-turbo")

# 2. Define a clear and focused agent
planner_writer_agent = Agent(
    role='Article Planner and Writer',
    goal='Plan and then write a concise, engaging summary on a
specified topic.',
    backstory=(
        'You are an expert technical writer and content strategist. '
        'Your strength lies in creating a clear, actionable plan
before writing, '
        'ensuring the final summary is both informative and easy to
digest.'
    ),
    verbose=True,
    allow_delegation=False,
    llm=llm # Assign the specific LLM to the agent
)

# 3. Define a task with a more structured and specific expected
output
topic = "The importance of Reinforcement Learning in AI"
high_level_task = Task(
    description=(
        f"1. Create a bullet-point plan for a summary on the topic:
'{topic}'.\n"
        f"2. Write the summary based on your plan, keeping it around
200 words."
    ),
    expected_output=(
        "A final report containing two distinct sections:\n\n"
        "### Plan\n"
        "- A bulleted list outlining the main points of the
summary.\n\n"
        "### Summary\n"
        "- A concise and well-structured summary of the topic."
    ),
    agent=planner_writer_agent,
)

# Create the crew with a clear process
crew = Crew(
    agents=[planner_writer_agent],
    tasks=[high_level_task],
    process=Process.sequential,

```

```
)

# Execute the task
print("## Running the planning and writing task ##")
result = crew.kickoff()

print("\n\n---\n## Task Result ##\n---")
print(result)
```

This code uses the CrewAI library to create an AI agent that plans and writes a summary on a given topic. It starts by importing necessary libraries, including Crew.ai and langchain_openai, and loading environment variables from a .env file. A ChatOpenAI language model is explicitly defined for use with the agent. An Agent named planner_writer_agent is created with a specific role and goal: to plan and then write a concise summary. The agent's backstory emphasizes its expertise in planning and technical writing. A Task is defined with a clear description to first create a plan and then write a summary on the topic "The importance of Reinforcement Learning in AI", with a specific format for the expected output. A Crew is assembled with the agent and task, set to process them sequentially. Finally, the crew.kickoff() method is called to execute the defined task and the result is printed.

Google DeepResearch

Google Gemini DeepResearch (see Fig.1) is an agent-based system designed for autonomous information retrieval and synthesis. It functions through a multi-step agentic pipeline that dynamically and iteratively queries Google Search to systematically explore complex topics. The system is engineered to process a large corpus of web-based sources, evaluate the collected data for relevance and knowledge gaps, and perform subsequent searches to address them. The final output consolidates the vetted information into a structured, multi-page summary with citations to the original sources.

Expanding on this, the system's operation is not a single query-response event but a managed, long-running process. It begins by deconstructing a user's prompt into a multi-point research plan (see Fig. 1), which is then presented to the user for review and modification. This allows for a collaborative shaping of the research trajectory before execution. Once the plan is approved, the agentic pipeline initiates its iterative search-and-analysis loop. This involves more than just executing a series of predefined searches; the agent dynamically formulates and refines its queries based on the

information it gathers, actively identifying knowledge gaps, corroborating data points, and resolving discrepancies.

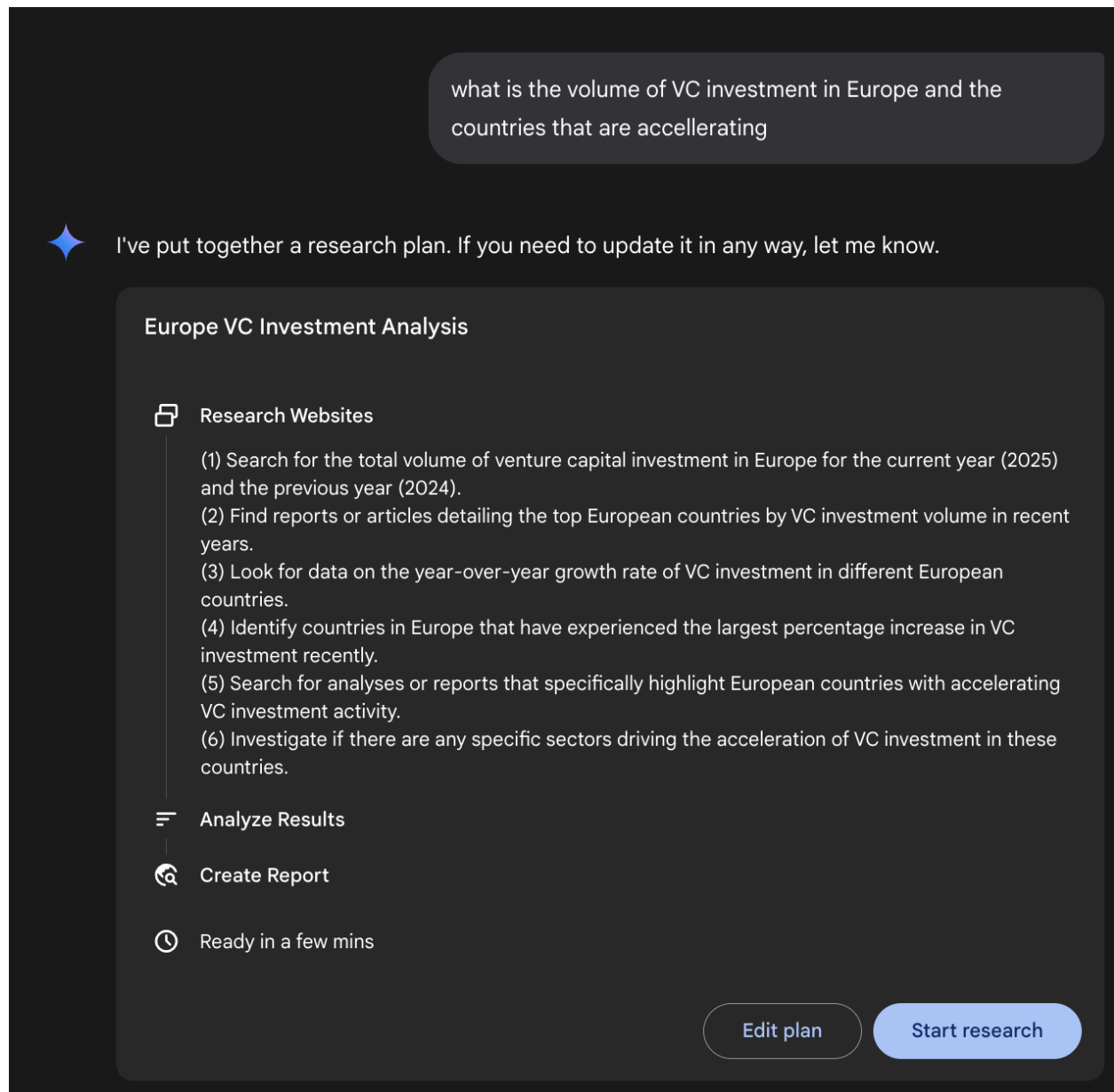


Fig. 1: Google Deep Research agent generating an execution plan for using Google Search as a tool.

A key architectural component is the system's ability to manage this process asynchronously. This design ensures that the investigation, which can involve analyzing hundreds of sources, is resilient to single-point failures and allows the user to disengage and be notified upon completion. The system can also integrate

user-provided documents, combining information from private sources with its web-based research. The final output is not merely a concatenated list of findings but a structured, multi-page report. During the synthesis phase, the model performs a critical evaluation of the collected information, identifying major themes and organizing the content into a coherent narrative with logical sections. The report is designed to be interactive, often including features like an audio overview, charts, and links to the original cited sources, allowing for verification and further exploration by the user. In addition to the synthesized results, the model explicitly returns the full list of sources it searched and consulted (see Fig.2). These are presented as citations, providing complete transparency and direct access to the primary information. This entire process transforms a simple query into a comprehensive, synthesized body of knowledge.

Europe VC Investment Analysis

Show thinking

✕

Initial Investment Landscape

I've started looking into the venture capital investment scene in Europe. My initial focus is on understanding the overall investment amounts for the current year and the previous year to establish a baseline.

Identifying Key Players

I'm now shifting my attention to pinpointing which European countries are currently leading in terms of attracting the highest volumes of venture capital. This will help me understand the major hubs of investment activity.

Focusing on Growth

To understand which countries are 'accelerating', I'm now looking for data that compares the growth rate of VC investments across different European nations on a year-over-year basis. This will highlight the regions experiencing the most dynamic changes.

Next Steps: Deeper Dive

My next step involves specifically searching for reports that identify countries with the largest percentage increases in VC investment recently. I also want to find analyses that directly discuss the concept of 'accelerating' VC investment within specific European countries to get a more nuanced understanding.

Researching websites

dealroom.co European Tec...

news.crunchbase.com Eur...

news.crunchbase.com ne...

startupsavant.com Countr...

seedblink.com State of Fu...

portugalbusinessnews.com

stateofeuropentech.com T...

vestbee.com CEE compani...

pitchbook.com Hot or not: ...

femaleswitch.com Top 20 ...

affinity.co Previewing the ...

innovatefinance.com UK Fi...

investeurope.eu atomico-s...

stateofeuropentech.com I...

sifted.eu These were the 1...

eif.org EIF VC Survey 2023...

news.crunchbase.com Eur...

svb.com Which countries ...

dealroom.co Europe Q2 20...

cbinsights.com VC Contin...

pitchbook.com Hot or not: ...

dealroom.co The State of ...

investeurope.eu Annual ac...

svb.com State of the Mark...

static.fintek.pl Europe Q3 ...

bebeez.eu The 10 largest ...

dealroom.co Europe | Deal...

investeurope.eu Investme...

investeurope.eu €143 billio...

dealroom.co Deep Tech: E...

lazard.com 2024 European...

vestbee.com 60 VC funds i...

eiturbanmobility.eu Europ...

hackernoon.com Most Pro...

therecursive.com Investm...

euractiv.com Fuelling Euro...

euractiv.com Europe's star...

wise.com Venture Capital ...

kpmg.com Q4'24 Venture ...

seedblink.com State of Fu...

bain.com Global Venture C...

visible.vc 12 Most Active V...

leadloft.com Hottest Tech ...

reddit.com What industrie...

sifted.eu Latest Venture C...

news.crunchbase.com The...

kpmg.com UK leads ventur...

bvkap.de 1Q 2025 Global V...

trustventure.de European ...

vestbee.com European VC ...

kpmg.com Record high on ...

bvkap.de 2024 Global Vent...

Fig. 2: An example of Deep Research plan being executed, resulting in Google Search being used as a tool to search various web sources.

By mitigating the substantial time and resource investment required for manual data acquisition and synthesis, Gemini DeepResearch provides a more structured and exhaustive method for information discovery. The system's value is particularly evident in complex, multi-faceted research tasks across various domains.

For instance, in competitive analysis, the agent can be directed to systematically gather and collate data on market trends, competitor product specifications, public sentiment from diverse online sources, and marketing strategies. This automated process replaces the laborious task of manually tracking multiple competitors, allowing analysts to focus on higher-order strategic interpretation rather than data collection (see Fig. 3).

 Europe VC Investment Analysis Export to Docs   

European Venture Capital Investment Landscape: Trends and Accelerating Countries

Executive Summary

This report provides a comprehensive analysis of the venture capital (VC) investment landscape in Europe, focusing on the trends observed in 2024 and the first quarter of 2025. The analysis identifies the total VC investment volume in Europe during these periods, highlights the top-performing countries, and pinpoints those experiencing an acceleration in investment activity. Furthermore, the report examines the key sectors that are attracting the most significant capital, offering insights into the evolving dynamics of the European venture ecosystem. Findings indicate a stabilization of the market in early 2025 after a mixed performance in 2024, with specific countries like Spain, the UK & Ireland, and the Netherlands demonstrating notable growth. The leading sectors driving investment include Artificial Intelligence and Machine Learning, Healthcare and Biotech, Fintech, Cleantech and Energy, and Deep Tech and Defense Tech.

State of European VC Investment in 2024

Total VC Investment Volume in Europe in 2024

The total volume of venture capital investment in Europe during 2024 presents a varied picture depending on the data source. Crunchbase data indicates a total of \$51 billion invested, representing a 5% decrease compared to the \$54 billion recorded in 2023. In contrast, Invest Europe reported a strong rebound with €18 billion (approximately \$19.4 billion) invested, marking a substantial 26% increase year-over-year. This figure positions 2024 as the second-highest year for VC investment in Europe, surpassed only by the record levels of 2022. GoingVC data, however, suggests a more significant downturn, reporting \$10 billion in VC funding for Europe in 2024, a 39% decline from the previous year. PitchBook data estimates the total deal value at €57.3 billion (approximately \$61.8 billion), indicating a 6.7% drop from the €55.1 billion invested in 2023.

The discrepancies in these figures highlight the challenges inherent in tracking venture capital investment and likely stem from differences in methodologies, including the scope of what is considered VC investment, deal size thresholds, and the inclusion or exclusion of specific funding types or regions. The timing of data collection and reporting can also contribute to these variations. Despite these conflicting numbers, a broader trend emerges when comparing 2024 to previous years. While Crunchbase and PitchBook suggest a decrease or stabilization compared to 2023, Invest Europe points to a strong recovery. However, all sources generally agree that the investment volume in 2024 remained significantly below the peak observed in 2021. This indicates a market recalibration following a period of extraordinary growth. Dealroom data further supports this, showing a 37% drop in European VC investment in 2023 compared to 2022, suggesting that 2024, according to some reports, might be showing signs of a positive shift after a period of decline.

Top European Countries by VC Investment Volume in 2024

The United Kingdom consistently emerged as the leading country in Europe for venture capital investment in 2024. Crunchbase data reported

Fig. 3: Final output generated by the Google Deep Research agent, analyzing on our behalf sources obtained using Google Search as a tool.

Similarly, in academic exploration, the system serves as a powerful tool for conducting extensive literature reviews. It can identify and summarize foundational papers, trace the development of concepts across numerous publications, and map out emerging research fronts within a specific field, thereby accelerating the initial and most time-consuming phase of academic inquiry.

The efficiency of this approach stems from the automation of the iterative search-and-filter cycle, which is a core bottleneck in manual research. Comprehensiveness is achieved by the system's capacity to process a larger volume and variety of information sources than is typically feasible for a human researcher within a comparable timeframe. This broader scope of analysis helps to reduce the potential for selection bias and increases the likelihood of uncovering less obvious but potentially critical information, leading to a more robust and well-supported understanding of the subject matter.

OpenAI Deep Research API

The OpenAI Deep Research API is a specialized tool designed to automate complex research tasks. It utilizes an advanced, agentic model that can independently reason, plan, and synthesize information from real-world sources. Unlike a simple Q&A model, it takes a high-level query and autonomously breaks it down into sub-questions, performs web searches using its built-in tools, and delivers a structured, citation-rich final report. The API provides direct programmatic access to this entire process, using at the time of writing models like `o3-deep-research-2025-06-26` for high-quality synthesis and the faster `o4-mini-deep-research-2025-06-26` for latency-sensitive application

The Deep Research API is useful because it automates what would otherwise be hours of manual research, delivering professional-grade, data-driven reports suitable for informing business strategy, investment decisions, or policy recommendations. Its key benefits include:

- **Structured, Cited Output:** It produces well-organized reports with inline citations linked to source metadata, ensuring claims are verifiable and data-backed.

- **Transparency:** Unlike the abstracted process in ChatGPT, the API exposes all intermediate steps, including the agent's reasoning, the specific web search queries it executed, and any code it ran. This allows for detailed debugging, analysis, and a deeper understanding of how the final answer was constructed.
- **Extensibility:** It supports the Model Context Protocol (MCP), enabling developers to connect the agent to private knowledge bases and internal data sources, blending public web research with proprietary information.

To use the API, you send a request to the `client.responses.create` endpoint, specifying a model, an input prompt, and the tools the agent can use. The input typically includes a `system_message` that defines the agent's persona and desired output format, along with the `user_query`. You must also include the `web_search_preview` tool and can optionally add others like `code_interpreter` or custom MCP tools (see Chapter 10) for internal data.

```
from openai import OpenAI

# Initialize the client with your API key
client = OpenAI(api_key="YOUR_OPENAI_API_KEY")

# Define the agent's role and the user's research question
system_message = """You are a professional researcher preparing a
structured, data-driven report.
Focus on data-rich insights, use reliable sources, and include inline
citations."""
user_query = "Research the economic impact of semaglutide on global
healthcare systems."

# Create the Deep Research API call
response = client.responses.create(
    model="o3-deep-research-2025-06-26",
    input=[
        {
            "role": "developer",
            "content": [{"type": "input_text", "text": system_message}]
        },
        {
            "role": "user",
            "content": [{"type": "input_text", "text": user_query}]
        }
    ],
    reasoning={"summary": "auto"},
    tools=[{"type": "web_search_preview"}]
)
```

```

# Access and print the final report from the response
final_report = response.output[-1].content[0].text
print(final_report)

# --- ACCESS INLINE CITATIONS AND METADATA ---
print("--- CITATIONS ---")
annotations = response.output[-1].content[0].annotations

if not annotations:
    print("No annotations found in the report.")
else:
    for i, citation in enumerate(annotations):
        # The text span the citation refers to
        cited_text =
final_report[citation.start_index:citation.end_index]

        print(f"Citation {i+1}:")
        print(f"  Cited Text: {cited_text}")
        print(f"  Title: {citation.title}")
        print(f"  URL: {citation.url}")
        print(f"  Location: chars
{citation.start_index}-{citation.end_index}")
print("\n" + "="*50 + "\n")

# --- INSPECT INTERMEDIATE STEPS ---
print("--- INTERMEDIATE STEPS ---")

# 1. Reasoning Steps: Internal plans and summaries generated by the
model.
try:
    reasoning_step = next(item for item in response.output if
item.type == "reasoning")
    print("\n[Found a Reasoning Step]")
    for summary_part in reasoning_step.summary:
        print(f"  - {summary_part.text}")
except StopIteration:
    print("\nNo reasoning steps found.")

# 2. Web Search Calls: The exact search queries the agent executed.
try:
    search_step = next(item for item in response.output if item.type
== "web_search_call")
    print("\n[Found a Web Search Call]")
    print(f"  Query Executed: '{search_step.action['query']}'")
    print(f"  Status: {search_step.status}")
except StopIteration:

```

```

        print("\nNo web search steps found.")

# 3. Code Execution: Any code run by the agent using the code
interpreter.
try:
    code_step = next(item for item in response.output if item.type ==
"code_interpreter_call")
    print("\n[Found a Code Execution Step]")
    print("  Code Input:")
    print(f"    ``python\n{code_step.input}\n    ``")
    print("  Code Output:")
    print(f"    {code_step.output}")
except StopIteration:
    print("\nNo code execution steps found.")

```

This code snippet utilizes the OpenAI API to perform a "Deep Research" task. It starts by initializing the OpenAI client with your API key, which is crucial for authentication. Then, it defines the role of the AI agent as a professional researcher and sets the user's research question about the economic impact of semaglutide. The code constructs an API call to the o3-deep-research-2025-06-26 model, providing the defined system message and user query as input. It also requests an automatic summary of the reasoning and enables web search capabilities. After making the API call, it extracts and prints the final generated report.

Subsequently, it attempts to access and display inline citations and metadata from the report's annotations, including the cited text, title, URL, and location within the report. Finally, it inspects and prints details about the intermediate steps the model took, such as reasoning steps, web search calls (including the query executed), and any code execution steps if a code interpreter was used.

At a Glance

What: Complex problems often cannot be solved with a single action and require foresight to achieve a desired outcome. Without a structured approach, an agentic system struggles to handle multifaceted requests that involve multiple steps and dependencies. This makes it difficult to break down high-level objectives into a manageable series of smaller, executable tasks. Consequently, the system fails to strategize effectively, leading to incomplete or incorrect results when faced with intricate goals.

Why: The Planning pattern offers a standardized solution by having an agentic system first create a coherent plan to address a goal. It involves decomposing a high-level objective into a sequence of smaller, actionable steps or sub-goals. This allows the system to manage complex workflows, orchestrate various tools, and handle dependencies in a logical order. LLMs are particularly well-suited for this, as they can generate plausible and effective plans based on their vast training data. This structured approach transforms a simple reactive agent into a strategic executor that can proactively work towards a complex objective and even adapt its plan if necessary.

Rule of thumb: Use this pattern when a user's request is too complex to be handled by a single action or tool. It is ideal for automating multi-step processes, such as generating a detailed research report, onboarding a new employee, or executing a competitive analysis. Apply the Planning pattern whenever a task requires a sequence of interdependent operations to reach a final, synthesized outcome.

Visual summary

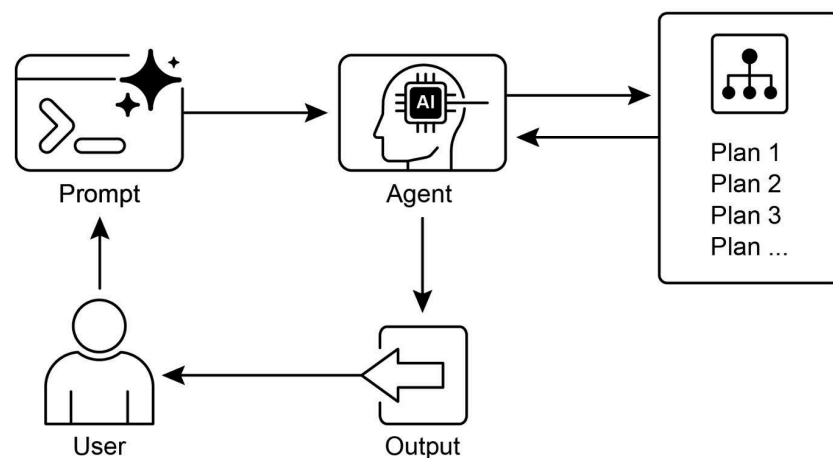


Fig.4; Planning design pattern

Key Takeaways

- Planning enables agents to break down complex goals into actionable, sequential steps.
- It is essential for handling multi-step tasks, workflow automation, and navigating complex environments.
- LLMs can perform planning by generating step-by-step approaches based on task descriptions.
- Explicitly prompting or designing tasks to require planning steps encourages this behavior in agent frameworks.
- Google Deep Research is an agent analyzing on our behalf sources obtained using Google Search as a tool. It reflects, plans, and executes

Conclusion

In conclusion, the Planning pattern is a foundational component that elevates agentic systems from simple reactive responders to strategic, goal-oriented executors. Modern large language models provide the core capability for this, autonomously decomposing high-level objectives into coherent, actionable steps. This pattern scales from straightforward, sequential task execution, as demonstrated by the CrewAI agent creating and following a writing plan, to more complex and dynamic systems. The Google DeepResearch agent exemplifies this advanced application, creating iterative research plans that adapt and evolve based on continuous information gathering. Ultimately, planning provides the essential bridge between human intent and automated execution for complex problems. By structuring a problem-solving approach, this pattern enables agents to manage intricate workflows and deliver comprehensive, synthesized results.

References

1. Google DeepResearch (Gemini Feature): gemini.google.com
2. OpenAI ,Introducing deep research <https://openai.com/index/introducing-deep-research/>
3. Perplexity, Introducing Perplexity Deep Research, <https://www.perplexity.ai/hub/blog/introducing-perplexity-deep-research>

Chapter 7: Multi-Agent Collaboration

While a monolithic agent architecture can be effective for well-defined problems, its capabilities are often constrained when faced with complex, multi-domain tasks. The Multi-Agent Collaboration pattern addresses these limitations by structuring a system as a cooperative ensemble of distinct, specialized agents. This approach is predicated on the principle of task decomposition, where a high-level objective is broken down into discrete sub-problems. Each sub-problem is then assigned to an agent possessing the specific tools, data access, or reasoning capabilities best suited for that task.

For example, a complex research query might be decomposed and assigned to a Research Agent for information retrieval, a Data Analysis Agent for statistical processing, and a Synthesis Agent for generating the final report. The efficacy of such a system is not merely due to the division of labor but is critically dependent on the mechanisms for inter-agent communication. This requires a standardized communication protocol and a shared ontology, allowing agents to exchange data, delegate sub-tasks, and coordinate their actions to ensure the final output is coherent.

This distributed architecture offers several advantages, including enhanced modularity, scalability, and robustness, as the failure of a single agent does not necessarily cause a total system failure. The collaboration allows for a synergistic outcome where the collective performance of the multi-agent system surpasses the potential capabilities of any single agent within the ensemble.

Multi-Agent Collaboration Pattern Overview

The Multi-Agent Collaboration pattern involves designing systems where multiple independent or semi-independent agents work together to achieve a common goal. Each agent typically has a defined role, specific goals aligned with the overall objective, and potentially access to different tools or knowledge bases. The power of this pattern lies in the interaction and synergy between these agents.

Collaboration can take various forms:

- **Sequential Handoffs:** One agent completes a task and passes its output to another agent for the next step in a pipeline (similar to the Planning pattern, but explicitly involving different agents).

- **Parallel Processing:** Multiple agents work on different parts of a problem simultaneously, and their results are later combined.
- **Debate and Consensus:** Multi-Agent Collaboration where Agents with varied perspectives and information sources engage in discussions to evaluate options, ultimately reaching a consensus or a more informed decision.
- **Hierarchical Structures:** A manager agent might delegate tasks to worker agents dynamically based on their tool access or plugin capabilities and synthesize their results. Each agent can also handle relevant groups of tools, rather than a single agent handling all the tools.
- **Expert Teams:** Agents with specialized knowledge in different domains (e.g., a researcher, a writer, an editor) collaborate to produce a complex output.
- **Critic-Reviewer:** Agents create initial outputs such as plans, drafts, or answers. A second group of agents then critically assesses this output for adherence to policies, security, compliance, correctness, quality, and alignment with organizational objectives. The original creator or a final agent revises the output based on this feedback. This pattern is particularly effective for code generation, research writing, logic checking, and ensuring ethical alignment. The advantages of this approach include increased robustness, improved quality, and a reduced likelihood of hallucinations or errors.

A multi-agent system (see Fig.1) fundamentally comprises the delineation of agent roles and responsibilities, the establishment of communication channels through which agents exchange information, and the formulation of a task flow or interaction protocol that directs their collaborative endeavors.

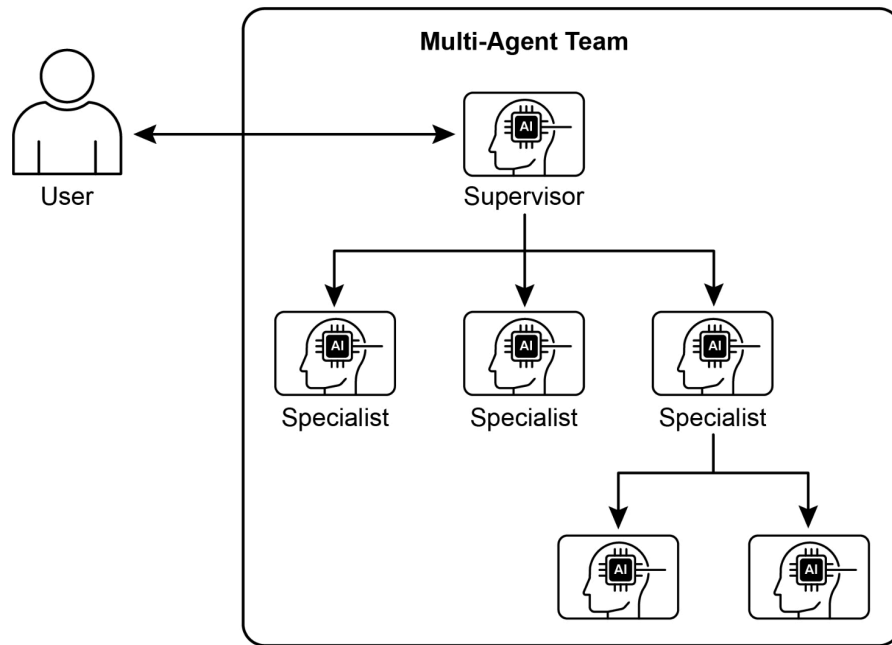


Fig.1: Example of multi-agent system

Frameworks such as Crew AI and Google ADK are engineered to facilitate this paradigm by providing structures for the specification of agents, tasks, and their interactive procedures. This approach is particularly effective for challenges necessitating a variety of specialized knowledge, encompassing multiple discrete phases, or leveraging the advantages of concurrent processing and the corroboration of information across agents.

Practical Applications & Use Cases

Multi-Agent Collaboration is a powerful pattern applicable across numerous domains:

- **Complex Research and Analysis:** A team of agents could collaborate on a research project. One agent might specialize in searching academic databases, another in summarizing findings, a third in identifying trends, and a fourth in synthesizing the information into a report. This mirrors how a human research team might operate.
- **Software Development:** Imagine agents collaborating on building software. One agent could be a requirements analyst, another a code generator, a third a tester,

and a fourth a documentation writer. They could pass outputs between each other to build and verify components.

- **Creative Content Generation:** Creating a marketing campaign could involve a market research agent, a copywriter agent, a graphic design agent (using image generation tools), and a social media scheduling agent, all working together.
- **Financial Analysis:** A multi-agent system could analyze financial markets. Agents might specialize in fetching stock data, analyzing news sentiment, performing technical analysis, and generating investment recommendations.
- **Customer Support Escalation:** A front-line support agent could handle initial queries, escalating complex issues to a specialist agent (e.g., a technical expert or a billing specialist) when needed, demonstrating a sequential handoff based on problem complexity.
- **Supply Chain Optimization:** Agents could represent different nodes in a supply chain (suppliers, manufacturers, distributors) and collaborate to optimize inventory levels, logistics, and scheduling in response to changing demand or disruptions.
- **Network Analysis & Remediation:** Autonomous operations benefit greatly from an agentic architecture, particularly in failure pinpointing. Multiple agents can collaborate to triage and remediate issues, suggesting optimal actions. These agents can also integrate with traditional machine learning models and tooling, leveraging existing systems while simultaneously offering the advantages of Generative AI.

The capacity to delineate specialized agents and meticulously orchestrate their interrelationships empowers developers to construct systems exhibiting enhanced modularity, scalability, and the ability to address complexities that would prove insurmountable for a singular, integrated agent.

Multi-Agent Collaboration: Exploring Interrelationships and Communication Structures

Understanding the intricate ways in which agents interact and communicate is fundamental to designing effective multi-agent systems. As depicted in Fig. 2, a spectrum of interrelationship and communication models exists, ranging from the simplest single-agent scenario to complex, custom-designed collaborative frameworks. Each model presents unique advantages and challenges, influencing the overall efficiency, robustness, and adaptability of the multi-agent system.

1. Single Agent: At the most basic level, a "Single Agent" operates autonomously without direct interaction or communication with other entities. While this model is straightforward to implement and manage, its capabilities are inherently limited by the individual agent's scope and resources. It is suitable for tasks that are decomposable into independent sub-problems, each solvable by a single, self-sufficient agent.

2. Network: The "Network" model represents a significant step towards collaboration, where multiple agents interact directly with each other in a decentralized fashion. Communication typically occurs peer-to-peer, allowing for the sharing of information, resources, and even tasks. This model fosters resilience, as the failure of one agent does not necessarily cripple the entire system. However, managing communication overhead and ensuring coherent decision-making in a large, unstructured network can be challenging.

3. Supervisor: In the "Supervisor" model, a dedicated agent, the "supervisor," oversees and coordinates the activities of a group of subordinate agents. The supervisor acts as a central hub for communication, task allocation, and conflict resolution. This hierarchical structure offers clear lines of authority and can simplify management and control. However, it introduces a single point of failure (the supervisor) and can become a bottleneck if the supervisor is overwhelmed by a large number of subordinates or complex tasks.

4. Supervisor as a Tool: This model is a nuanced extension of the "Supervisor" concept, where the supervisor's role is less about direct command and control and more about providing resources, guidance, or analytical support to other agents. The supervisor might offer tools, data, or computational services that enable other agents to perform their tasks more effectively, without necessarily dictating their every action. This approach aims to leverage the supervisor's capabilities without imposing rigid top-down control.

5. Hierarchical: The "Hierarchical" model expands upon the supervisor concept to create a multi-layered organizational structure. This involves multiple levels of supervisors, with higher-level supervisors overseeing lower-level ones, and ultimately, a collection of operational agents at the lowest tier. This structure is well-suited for complex problems that can be decomposed into sub-problems, each managed by a specific layer of the hierarchy. It provides a structured approach to scalability and complexity management, allowing for distributed decision-making within defined boundaries.

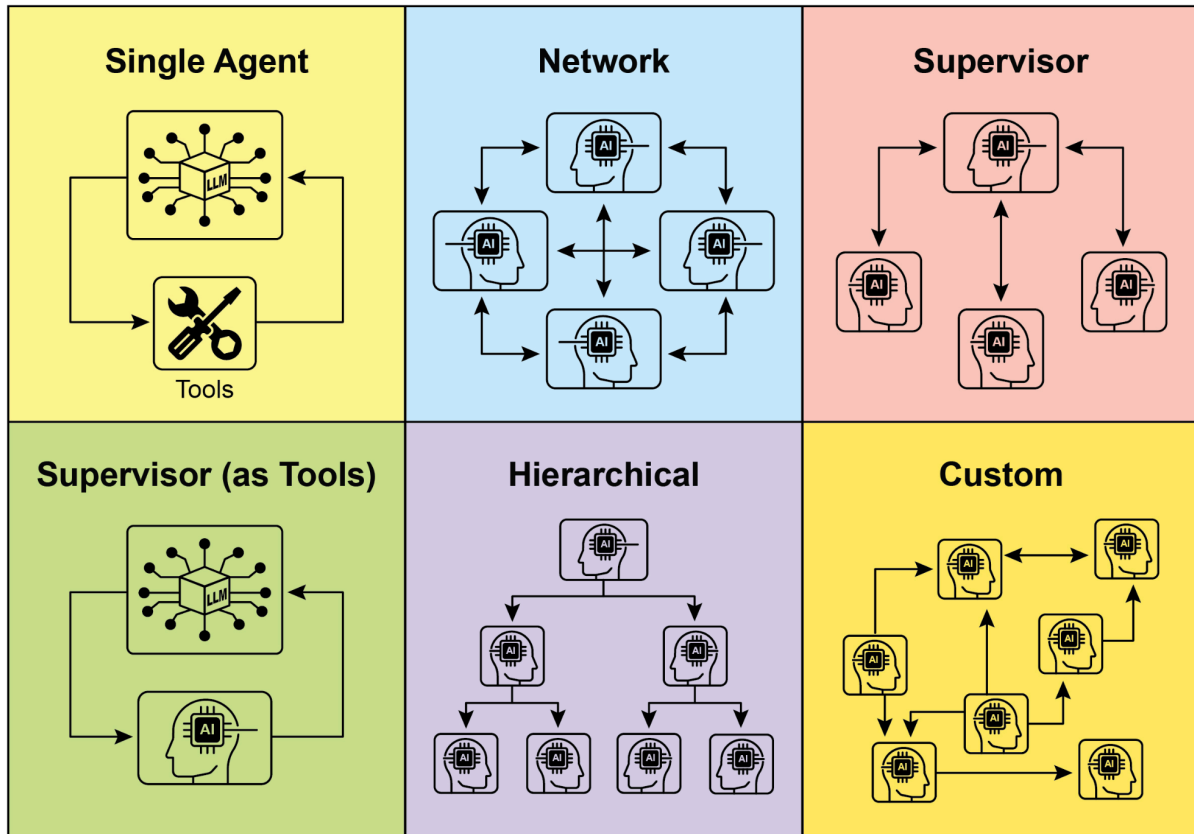


Fig. 2: Agents communicate and interact in various ways.

6. Custom: The "Custom" model represents the ultimate flexibility in multi-agent system design. It allows for the creation of unique interrelationship and communication structures tailored precisely to the specific requirements of a given problem or application. This can involve hybrid approaches that combine elements from the previously mentioned models, or entirely novel designs that emerge from the unique constraints and opportunities of the environment. Custom models often arise from the need to optimize for specific performance metrics, handle highly dynamic environments, or incorporate domain-specific knowledge into the system's architecture. Designing and implementing custom models typically requires a deep understanding of multi-agent systems principles and careful consideration of communication protocols, coordination mechanisms, and emergent behaviors.

In summary, the choice of interrelationship and communication model for a multi-agent system is a critical design decision. Each model offers distinct advantages and disadvantages, and the optimal choice depends on factors such as the complexity of the task, the number of agents, the desired level of autonomy, the need

for robustness, and the acceptable communication overhead. Future advancements in multi-agent systems will likely continue to explore and refine these models, as well as develop new paradigms for collaborative intelligence.

Hands-On code (Crew AI)

This Python code defines an AI-powered crew using the CrewAI framework to generate a blog post about AI trends. It starts by setting up the environment, loading API keys from a .env file. The core of the application involves defining two agents: a researcher to find and summarize AI trends, and a writer to create a blog post based on the research.

Two tasks are defined accordingly: one for researching the trends and another for writing the blog post, with the writing task depending on the output of the research task. These agents and tasks are then assembled into a Crew, specifying a sequential process where tasks are executed in order. The Crew is initialized with the agents, tasks, and a language model (specifically the "gemini-2.0-flash" model). The main function executes this crew using the kickoff() method, orchestrating the collaboration between the agents to produce the desired output. Finally, the code prints the final result of the crew's execution, which is the generated blog post.

```
import os
from dotenv import load_dotenv
from crewai import Agent, Task, Crew, Process
from langchain_google_genai import ChatGoogleGenerativeAI

def setup_environment():
    """Loads environment variables and checks for the required API
    key."""
    load_dotenv()
    if not os.getenv("GOOGLE_API_KEY"):
        raise ValueError("GOOGLE_API_KEY not found. Please set it in
        your .env file.")

def main():
    """
    Initializes and runs the AI crew for content creation using the
    latest Gemini model.
    """
    setup_environment()

    # Define the language model to use.
```

```

# Updated to a model from the Gemini 2.0 series for better
performance and features.
# For cutting-edge (preview) capabilities, you could use
"gemini-2.5-flash".
llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash")

# Define Agents with specific roles and goals
researcher = Agent(
    role='Senior Research Analyst',
    goal='Find and summarize the latest trends in AI.',
    backstory="You are an experienced research analyst with a
knack for identifying key trends and synthesizing information.",
    verbose=True,
    allow_delegation=False,
)

writer = Agent(
    role='Technical Content Writer',
    goal='Write a clear and engaging blog post based on research
findings.',
    backstory="You are a skilled writer who can translate complex
technical topics into accessible content.",
    verbose=True,
    allow_delegation=False,
)

# Define Tasks for the agents
research_task = Task(
    description="Research the top 3 emerging trends in Artificial
Intelligence in 2024-2025. Focus on practical applications and
potential impact.",
    expected_output="A detailed summary of the top 3 AI trends,
including key points and sources.",
    agent=researcher,
)

writing_task = Task(
    description="Write a 500-word blog post based on the research
findings. The post should be engaging and easy for a general audience
to understand.",
    expected_output="A complete 500-word blog post about the
latest AI trends.",
    agent=writer,
    context=[research_task],
)

# Create the Crew

```

```

blog_creation_crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, writing_task],
    process=Process.sequential,
    llm=llm,
    verbose=2 # Set verbosity for detailed crew execution logs
)

# Execute the Crew
print("## Running the blog creation crew with Gemini 2.0 Flash...
##")
try:
    result = blog_creation_crew.kickoff()
    print("\n-----\n")
    print("## Crew Final Output ##")
    print(result)
except Exception as e:
    print(f"\nAn unexpected error occurred: {e}")

if __name__ == "__main__":
    main()

```

We will now delve into further examples within the Google ADK framework, with particular emphasis on hierarchical, parallel, and sequential coordination paradigms, alongside the implementation of an agent as an operational instrument.

Hands-on Code (Google ADK)

The following code example demonstrates the establishment of a hierarchical agent structure within the Google ADK through the creation of a parent-child relationship. The code defines two types of agents: LlmAgent and a custom TaskExecutor agent derived from BaseAgent. The TaskExecutor is designed for specific, non-LLM tasks and in this example, it simply yields a "Task finished successfully" event. An LlmAgent named greeter is initialized with a specified model and instruction to act as a friendly greeter. The custom TaskExecutor is instantiated as task_doer. A parent LlmAgent called coordinator is created, also with a model and instructions. The coordinator's instructions guide it to delegate greetings to the greeter and task execution to the task_doer. The greeter and task_doer are added as sub-agents to the coordinator, establishing a parent-child relationship. The code then asserts that this relationship is correctly set up. Finally, it prints a message indicating that the agent hierarchy has been successfully created.

```

from google.adk.agents import LlmAgent, BaseAgent
from google.adk.agents.invocation_context import InvocationContext
from google.adk.events import Event
from typing import AsyncGenerator

# Correctly implement a custom agent by extending BaseAgent
class TaskExecutor(BaseAgent):
    """A specialized agent with custom, non-LLM behavior."""
    name: str = "TaskExecutor"
    description: str = "Executes a predefined task."

    async def _run_async_impl(self, context: InvocationContext) ->
    AsyncGenerator[Event, None]:
        """Custom implementation logic for the task."""
        # This is where your custom logic would go.
        # For this example, we'll just yield a simple event.
        yield Event(author=self.name, content="Task finished
successfully.")

# Define individual agents with proper initialization
# LlmAgent requires a model to be specified.
greeter = LlmAgent(
    name="Greeter",
    model="gemini-2.0-flash-exp",
    instruction="You are a friendly greeter."
)
task_doer = TaskExecutor() # Instantiate our concrete custom agent

# Create a parent agent and assign its sub-agents
# The parent agent's description and instructions should guide its
delegation logic.
coordinator = LlmAgent(
    name="Coordinator",
    model="gemini-2.0-flash-exp",
    description="A coordinator that can greet users and execute
tasks.",
    instruction="When asked to greet, delegate to the Greeter. When
asked to perform a task, delegate to the TaskExecutor.",
    sub_agents=[
        greeter,
        task_doer
    ]
)

# The ADK framework automatically establishes the parent-child

```

```
relationships.
# These assertions will pass if checked after initialization.
assert greeter.parent_agent == coordinator
assert task_doer.parent_agent == coordinator

print("Agent hierarchy created successfully.")
```

This code excerpt illustrates the employment of the LoopAgent within the Google ADK framework to establish iterative workflows. The code defines two agents: ConditionChecker and ProcessingStep. ConditionChecker is a custom agent that checks a "status" value in the session state. If the "status" is "completed", ConditionChecker escalates an event to stop the loop. Otherwise, it yields an event to continue the loop. ProcessingStep is an LlmAgent using the "gemini-2.0-flash-exp" model. Its instruction is to perform a task and set the session "status" to "completed" if it's the final step. A LoopAgent named StatusPoller is created. StatusPoller is configured with max_iterations=10. StatusPoller includes both ProcessingStep and an instance of ConditionChecker as sub-agents. The LoopAgent will execute the sub-agents sequentially for up to 10 iterations, stopping if ConditionChecker finds the status is "completed".

```
import asyncio
from typing import AsyncGenerator
from google.adk.agents import LoopAgent, LlmAgent, BaseAgent
from google.adk.events import Event, EventActions
from google.adk.agents.invocation_context import InvocationContext

# Best Practice: Define custom agents as complete, self-describing
classes.
class ConditionChecker(BaseAgent):
    """A custom agent that checks for a 'completed' status in the
    session state."""
    name: str = "ConditionChecker"
    description: str = "Checks if a process is complete and signals
    the loop to stop."

    async def _run_async_impl(
        self, context: InvocationContext
    ) -> AsyncGenerator[Event, None]:
        """Checks state and yields an event to either continue or stop
        the loop."""
        status = context.session.state.get("status", "pending")
```

```

        is_done = (status == "completed")

        if is_done:
            # Escalate to terminate the loop when the condition is
            met.
            yield Event(author=self.name,
            actions=EventActions(escalate=True))
        else:
            # Yield a simple event to continue the loop.
            yield Event(author=self.name, content="Condition not met,
            continuing loop.")

# Correction: The LlmAgent must have a model and clear instructions.
process_step = LlmAgent(
    name="ProcessingStep",
    model="gemini-2.0-flash-exp",
    instruction="You are a step in a longer process. Perform your
    task. If you are the final step, update session state by setting
    'status' to 'completed'."
)

# The LoopAgent orchestrates the workflow.
poller = LoopAgent(
    name="StatusPoller",
    max_iterations=10,
    sub_agents=[
        process_step,
        ConditionChecker() # Instantiating the well-defined custom
agent.
    ]
)

# This poller will now execute 'process_step'
# and then 'ConditionChecker'
# repeatedly until the status is 'completed' or 10 iterations
# have passed.

```

This code excerpt elucidates the SequentialAgent pattern within the Google ADK, engineered for the construction of linear workflows. This code defines a sequential agent pipeline using the google.adk.agents library. The pipeline consists of two agents, step1 and step2. step1 is named "Step1_Fetch" and its output will be stored in the session state under the key "data". step2 is named "Step2_Process" and is instructed to analyze the information stored in session.state["data"] and provide a summary. The SequentialAgent named "MyPipeline" orchestrates the execution of

these sub-agents. When the pipeline is run with an initial input, step1 will execute first. The response from step1 will be saved into the session state under the key "data". Subsequently, step2 will execute, utilizing the information that step1 placed into the state as per its instruction. This structure allows for building workflows where the output of one agent becomes the input for the next. This is a common pattern in creating multi-step AI or data processing pipelines.

```
from google.adk.agents import SequentialAgent, Agent

# This agent's output will be saved to session.state["data"]
step1 = Agent(name="Step1_Fetch", output_key="data")

# This agent will use the data from the previous step.
# We instruct it on how to find and use this data.
step2 = Agent(
    name="Step2_Process",
    instruction="Analyze the information found in state['data'] and
provide a summary."
)

pipeline = SequentialAgent(
    name="MyPipeline",
    sub_agents=[step1, step2]
)

# When the pipeline is run with an initial input, Step1 will execute,
# its response will be stored in session.state["data"], and then
# Step2 will execute, using the information from the state as
instructed.
```

The following code example illustrates the ParallelAgent pattern within the Google ADK, which facilitates the concurrent execution of multiple agent tasks. The data_gatherer is designed to run two sub-agents concurrently: weather_fetcher and news_fetcher. The weather_fetcher agent is instructed to get the weather for a given location and store the result in session.state["weather_data"]. Similarly, the news_fetcher agent is instructed to retrieve the top news story for a given topic and store it in session.state["news_data"]. Each sub-agent is configured to use the "gemini-2.0-flash-exp" model. The ParallelAgent orchestrates the execution of these sub-agents, allowing them to work in parallel. The results from both weather_fetcher and news_fetcher would be gathered and stored in the session state. Finally, the

example shows how to access the collected weather and news data from the `final_state` after the agent's execution is complete.

```
from google.adk.agents import Agent, ParallelAgent

# It's better to define the fetching logic as tools for the agents
# For simplicity in this example, we'll embed the logic in the
agent's instruction.
# In a real-world scenario, you would use tools.

# Define the individual agents that will run in parallel
weather_fetcher = Agent(
    name="weather_fetcher",
    model="gemini-2.0-flash-exp",
    instruction="Fetch the weather for the given location and return
only the weather report.",
    output_key="weather_data" # The result will be stored in
session.state["weather_data"]
)

news_fetcher = Agent(
    name="news_fetcher",
    model="gemini-2.0-flash-exp",
    instruction="Fetch the top news story for the given topic and
return only that story.",
    output_key="news_data" # The result will be stored in
session.state["news_data"]
)

# Create the ParallelAgent to orchestrate the sub-agents
data_gatherer = ParallelAgent(
    name="data_gatherer",
    sub_agents=[
        weather_fetcher,
        news_fetcher
    ]
)
```

The provided code segment exemplifies the "Agent as a Tool" paradigm within the Google ADK, enabling an agent to utilize the capabilities of another agent in a manner analogous to function invocation. Specifically, the code defines an image generation system using Google's `LlmAgent` and `AgentTool` classes. It consists of two agents: a parent `artist_agent` and a sub-agent `image_generator_agent`. The `generate_image`

function is a simple tool that simulates image creation, returning mock image data. The `image_generator_agent` is responsible for using this tool based on a text prompt it receives. The `artist_agent`'s role is to first invent a creative image prompt. It then calls the `image_generator_agent` through an `AgentTool` wrapper. The `AgentTool` acts as a bridge, allowing one agent to use another agent as a tool. When the `artist_agent` calls the `image_tool`, the `AgentTool` invokes the `image_generator_agent` with the artist's invented prompt. The `image_generator_agent` then uses the `generate_image` function with that prompt. Finally, the generated image (or mock data) is returned back up through the agents. This architecture demonstrates a layered agent system where a higher-level agent orchestrates a lower-level, specialized agent to perform a task.

```
from google.adk.agents import LlmAgent
from google.adk.tools import agent_tool
from google.genai import types

# 1. A simple function tool for the core capability.
# This follows the best practice of separating actions from
reasoning.
def generate_image(prompt: str) -> dict:
    """
    Generates an image based on a textual prompt.

    Args:
        prompt: A detailed description of the image to generate.

    Returns:
        A dictionary with the status and the generated image bytes.
    """
    print(f"TOOL: Generating image for prompt: '{prompt}'")
    # In a real implementation, this would call an image generation
API.
    # For this example, we return mock image data.
    mock_image_bytes = b"mock_image_data_for_a_cat_wearing_a_hat"
    return {
        "status": "success",
        # The tool returns the raw bytes, the agent will handle the
Part creation.
        "image_bytes": mock_image_bytes,
        "mime_type": "image/png"
    }

# 2. Refactor the ImageGeneratorAgent into an LlmAgent.
# It now correctly uses the input passed to it.
```

```

image_generator_agent = LlmAgent(
    name="ImageGen",
    model="gemini-2.0-flash",
    description="Generates an image based on a detailed text prompt.",
    instruction=(
        "You are an image generation specialist. Your task is to take
the user's request "
        "and use the `generate_image` tool to create the image. "
        "The user's entire request should be used as the 'prompt'
argument for the tool. "
        "After the tool returns the image bytes, you MUST output the
image."
    ),
    tools=[generate_image]
)

# 3. Wrap the corrected agent in an AgentTool.
# The description here is what the parent agent sees.
image_tool = agent_tool.AgentTool(
    agent=image_generator_agent,
    description="Use this tool to generate an image. The input should
be a descriptive prompt of the desired image."
)

# 4. The parent agent remains unchanged. Its logic was correct.
artist_agent = LlmAgent(
    name="Artist",
    model="gemini-2.0-flash",
    instruction=(
        "You are a creative artist. First, invent a creative and
descriptive prompt for an image. "
        "Then, use the `ImageGen` tool to generate the image using
your prompt."
    ),
    tools=[image_tool]
)

```

At a Glance

What: Complex problems often exceed the capabilities of a single, monolithic LLM-based agent. A solitary agent may lack the diverse, specialized skills or access to the specific tools needed to address all parts of a multifaceted task. This limitation creates a bottleneck, reducing the system's overall effectiveness and scalability. As a

result, tackling sophisticated, multi-domain objectives becomes inefficient and can lead to incomplete or suboptimal outcomes.

Why: The Multi-Agent Collaboration pattern offers a standardized solution by creating a system of multiple, cooperating agents. A complex problem is broken down into smaller, more manageable sub-problems. Each sub-problem is then assigned to a specialized agent with the precise tools and capabilities required to solve it. These agents work together through defined communication protocols and interaction models like sequential handoffs, parallel workstreams, or hierarchical delegation. This agentic, distributed approach creates a synergistic effect, allowing the group to achieve outcomes that would be impossible for any single agent.

Rule of thumb: Use this pattern when a task is too complex for a single agent and can be decomposed into distinct sub-tasks requiring specialized skills or tools. It is ideal for problems that benefit from diverse expertise, parallel processing, or a structured workflow with multiple stages, such as complex research and analysis, software development, or creative content generation.

Visual summary

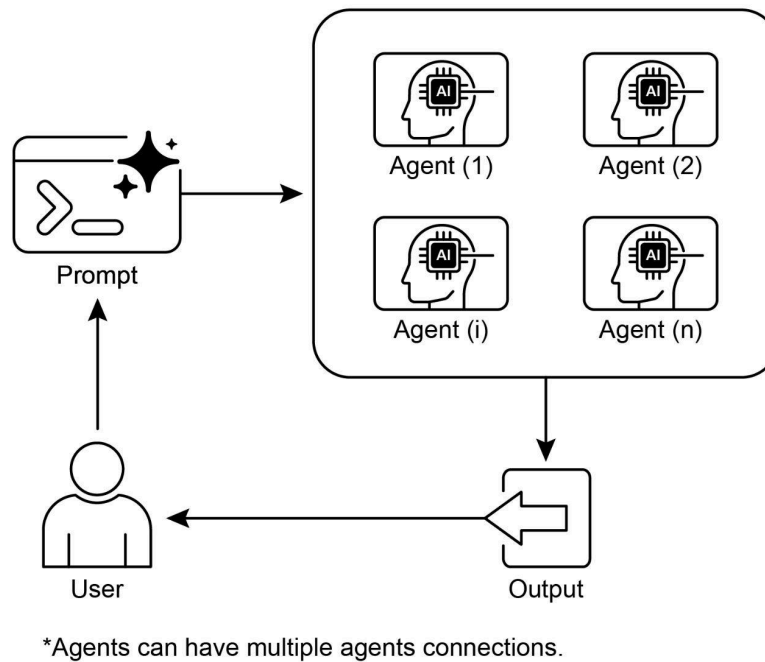


Fig.3: Multi-Agent design pattern

Key Takeaways

- Multi-agent collaboration involves multiple agents working together to achieve a common goal.
- This pattern leverages specialized roles, distributed tasks, and inter-agent communication.
- Collaboration can take forms like sequential handoffs, parallel processing, debate, or hierarchical structures.
- This pattern is ideal for complex problems requiring diverse expertise or multiple distinct stages.

Conclusion

This chapter explored the Multi-Agent Collaboration pattern, demonstrating the benefits of orchestrating multiple specialized agents within systems. We examined various collaboration models, emphasizing the pattern's essential role in addressing

complex, multifaceted problems across diverse domains. Understanding agent collaboration naturally leads to an inquiry into their interactions with the external environment.

References

1. Multi-Agent Collaboration Mechanisms: A Survey of LLMs,
<https://arxiv.org/abs/2501.06322>
2. Multi-Agent System — The Power of Collaboration,
<https://aravindakumar.medium.com/introducing-multi-agent-frameworks-the-power-of-collaboration-e9db31bba1b6>

Chapter 8: Memory Management

Effective memory management is crucial for intelligent agents to retain information. Agents require different types of memory, much like humans, to operate efficiently. This chapter delves into memory management, specifically addressing the immediate (short-term) and persistent (long-term) memory requirements of agents.

In agent systems, memory refers to an agent's ability to retain and utilize information from past interactions, observations, and learning experiences. This capability allows agents to make informed decisions, maintain conversational context, and improve over time. Agent memory is generally categorized into two main types:

- **Short-Term Memory (Contextual Memory):** Similar to working memory, this holds information currently being processed or recently accessed. For agents using large language models (LLMs), short-term memory primarily exists within the context window. This window contains recent messages, agent replies, tool usage results, and agent reflections from the current interaction, all of which inform the LLM's subsequent responses and actions. The context window has a limited capacity, restricting the amount of recent information an agent can directly access. Efficient short-term memory management involves keeping the most relevant information within this limited space, possibly through techniques like summarizing older conversation segments or emphasizing key details. The advent of models with 'long context' windows simply expands the size of this short-term memory, allowing more information to be held within a single interaction. However, this context is still ephemeral and is lost once the session concludes, and it can be costly and inefficient to process every time. Consequently, agents require separate memory types to achieve true persistence, recall information from past interactions, and build a lasting knowledge base.
- **Long-Term Memory (Persistent Memory):** This acts as a repository for information agents need to retain across various interactions, tasks, or extended periods, akin to long-term knowledge bases. Data is typically stored outside the agent's immediate processing environment, often in databases, knowledge graphs, or vector databases. In vector databases, information is converted into numerical vectors and stored, enabling agents to retrieve data based on semantic similarity rather than exact keyword matches, a process known as semantic search. When an agent needs information from long-term memory, it queries the external storage, retrieves relevant data, and integrates it into the short-term context for immediate use, thus combining prior

knowledge with the current interaction.

Practical Applications & Use Cases

Memory management is vital for agents to track information and perform intelligently over time. This is essential for agents to surpass basic question-answering capabilities. Applications include:

- **Chatbots and Conversational AI:** Maintaining conversation flow relies on short-term memory. Chatbots require remembering prior user inputs to provide coherent responses. Long-term memory enables chatbots to recall user preferences, past issues, or prior discussions, offering personalized and continuous interactions.
- **Task-Oriented Agents:** Agents managing multi-step tasks need short-term memory to track previous steps, current progress, and overall goals. This information might reside in the task's context or temporary storage. Long-term memory is crucial for accessing specific user-related data not in the immediate context.
- **Personalized Experiences:** Agents offering tailored interactions utilize long-term memory to store and retrieve user preferences, past behaviors, and personal information. This allows agents to adapt their responses and suggestions.
- **Learning and Improvement:** Agents can refine their performance by learning from past interactions. Successful strategies, mistakes, and new information are stored in long-term memory, facilitating future adaptations. Reinforcement learning agents store learned strategies or knowledge in this way.
- **Information Retrieval (RAG):** Agents designed for answering questions access a knowledge base, their long-term memory, often implemented within Retrieval Augmented Generation (RAG). The agent retrieves relevant documents or data to inform its responses.
- **Autonomous Systems:** Robots or self-driving cars require memory for maps, routes, object locations, and learned behaviors. This involves short-term memory for immediate surroundings and long-term memory for general environmental knowledge.

Memory enables agents to maintain history, learn, personalize interactions, and manage complex, time-dependent problems.

Hands-On Code: Memory Management in Google Agent Developer Kit (ADK)

The Google Agent Developer Kit (ADK) offers a structured method for managing context and memory, including components for practical application. A solid grasp of ADK's Session, State, and Memory is vital for building agents that need to retain information.

Just as in human interactions, agents require the ability to recall previous exchanges to conduct coherent and natural conversations. ADK simplifies context management through three core concepts and their associated services.

Every interaction with an agent can be considered a unique conversation thread. Agents might need to access data from earlier interactions. ADK structures this as follows:

- **Session:** An individual chat thread that logs messages and actions (Events) for that specific interaction, also storing temporary data (State) relevant to that conversation.
- **State (session.state):** Data stored within a Session, containing information relevant only to the current, active chat thread.
- **Memory:** A searchable repository of information sourced from various past chats or external sources, serving as a resource for data retrieval beyond the immediate conversation.

ADK provides dedicated services for managing critical components essential for building complex, stateful, and context-aware agents. The SessionService manages chat threads (Session objects) by handling their initiation, recording, and termination, while the MemoryService oversees the storage and retrieval of long-term knowledge (Memory).

Both the SessionService and MemoryService offer various configuration options, allowing users to choose storage methods based on application needs. In-memory options are available for testing purposes, though data will not persist across restarts. For persistent storage and scalability, ADK also supports database and cloud-based services.

Session: Keeping Track of Each Chat

A Session object in ADK is designed to track and manage individual chat threads. Upon initiation of a conversation with an agent, the SessionService generates a Session object, represented as `google.adk.sessions.Session`. This object encapsulates all data relevant to a specific conversation thread, including unique identifiers (id, app_name, user_id), a chronological record of events as Event objects, a storage area for session-specific temporary data known as state, and a timestamp indicating the last update (last_update_time). Developers typically interact with Session objects indirectly through the SessionService. The SessionService is responsible for managing the lifecycle of conversation sessions, which includes initiating new sessions, resuming previous sessions, recording session activity (including state updates), identifying active sessions, and managing the removal of session data. The ADK provides several SessionService implementations with varying storage mechanisms for session history and temporary data, such as the InMemorySessionService, which is suitable for testing but does not provide data persistence across application restarts.

```
# Example: Using InMemorySessionService
# This is suitable for local development and testing where data
# persistence across application restarts is not required.
from google.adk.sessions import InMemorySessionService
session_service = InMemorySessionService()
```

Then there's DatabaseSessionService if you want reliable saving to a database you manage.

```
# Example: Using DatabaseSessionService
# This is suitable for production or development requiring persistent
# storage.
# You need to configure a database URL (e.g., for SQLite, PostgreSQL,
# etc.).
# Requires: pip install google-adk[sqlalchemy] and a database driver
# (e.g., psycopg2 for PostgreSQL)
from google.adk.sessions import DatabaseSessionService
# Example using a local SQLite file:
db_url = "sqlite:///./my_agent_data.db"
session_service = DatabaseSessionService(db_url=db_url)
```

Besides, there's `VertexAiSessionService` which uses Vertex AI infrastructure for scalable production on Google Cloud.

```
# Example: Using VertexAiSessionService
# This is suitable for scalable production on Google Cloud Platform,
# leveraging
# Vertex AI infrastructure for session management.
# Requires: pip install google-adk[vertexai] and GCP
# setup/authentication
from google.adk.sessions import VertexAiSessionService

PROJECT_ID = "your-gcp-project-id" # Replace with your GCP project ID
LOCATION = "us-central1" # Replace with your desired GCP location
# The app_name used with this service should correspond to the
# Reasoning Engine ID or name
REASONING_ENGINE_APP_NAME =
"projects/your-gcp-project-id/locations/us-central1/reasoningEngines/
your-engine-id" # Replace with your Reasoning Engine resource name

session_service = VertexAiSessionService(project=PROJECT_ID,
location=LOCATION)
# When using this service, pass REASONING_ENGINE_APP_NAME to service
# methods:
# session_service.create_session(app_name=REASONING_ENGINE_APP_NAME,
# ...)
# session_service.get_session(app_name=REASONING_ENGINE_APP_NAME,
# ...)
# session_service.append_event(session, event,
# app_name=REASONING_ENGINE_APP_NAME)
# session_service.delete_session(app_name=REASONING_ENGINE_APP_NAME,
# ...)
```

Choosing an appropriate `SessionService` is crucial as it determines how the agent's interaction history and temporary data are stored and their persistence.

Each message exchange involves a cyclical process: A message is received, the Runner retrieves or establishes a Session using the `SessionService`, the agent processes the message using the Session's context (state and historical interactions), the agent generates a response and may update the state, the Runner encapsulates this as an Event, and the `session_service.append_event` method records the new

event and updates the state in storage. The Session then awaits the next message. Ideally, the `delete_session` method is employed to terminate the session when the interaction concludes. This process illustrates how the SessionService maintains continuity by managing the Session-specific history and temporary data.

State: The Session's Scratchpad

In the ADK, each Session, representing a chat thread, includes a state component akin to an agent's temporary working memory for the duration of that specific conversation. While `session.events` logs the entire chat history, `session.state` stores and updates dynamic data points relevant to the active chat.

Fundamentally, `session.state` operates as a dictionary, storing data as key-value pairs. Its core function is to enable the agent to retain and manage details essential for coherent dialogue, such as user preferences, task progress, incremental data collection, or conditional flags influencing subsequent agent actions.

The state's structure comprises string keys paired with values of serializable Python types, including strings, numbers, booleans, lists, and dictionaries containing these basic types. State is dynamic, evolving throughout the conversation. The permanence of these changes depends on the configured SessionService.

State organization can be achieved using key prefixes to define data scope and persistence. Keys without prefixes are session-specific.

- The user: prefix associates data with a user ID across all sessions.
- The app: prefix designates data shared among all users of the application.
- The temp: prefix indicates data valid only for the current processing turn and is not persistently stored.

The agent accesses all state data through a single `session.state` dictionary. The SessionService handles data retrieval, merging, and persistence. State should be updated upon adding an Event to the session history via `session_service.append_event()`. This ensures accurate tracking, proper saving in persistent services, and safe handling of state changes.

1. **The Simple Way: Using `output_key` (for Agent Text Replies):** This is the easiest method if you just want to save your agent's final text response directly into the state. When you set up your LlmAgent, just tell it the `output_key` you want to use. The Runner sees this and automatically creates the necessary actions to

save the response to the state when it appends the event. Let's look at a code example demonstrating state update via `output_key`.

```
# Import necessary classes from the Google Agent Developer Kit
(ADK)
from google.adk.agents import LlmAgent
from google.adk.sessions import InMemorySessionService, Session
from google.adk.runners import Runner
from google.genai.types import Content, Part

# Define an LlmAgent with an output_key.
greeting_agent = LlmAgent(
    name="Greeter",
    model="gemini-2.0-flash",
    instruction="Generate a short, friendly greeting.",
    output_key="last_greeting"
)

# --- Setup Runner and Session ---
app_name, user_id, session_id = "state_app", "user1", "session1"
session_service = InMemorySessionService()
runner = Runner(
    agent=greeting_agent,
    app_name=app_name,
    session_service=session_service
)
session = session_service.create_session(
    app_name=app_name,
    user_id=user_id,
    session_id=session_id
)

print(f"Initial state: {session.state}")

# --- Run the Agent ---
user_message = Content(parts=[Part(text="Hello")])
print("\n--- Running the agent ---")
for event in runner.run(
    user_id=user_id,
    session_id=session_id,
    new_message=user_message
):
    if event.is_final_response():
        print("Agent responded.")

# --- Check Updated State ---
# Correctly check the state *after* the runner has finished
```

```
processing all events.
updated_session = session_service.get_session(app_name, user_id,
session_id)
print(f"\nState after agent run: {updated_session.state}")
```

Behind the scenes, the Runner sees your `output_key` and automatically creates the necessary actions with a `state_delta` when it calls `append_event`.

- 2. The Standard Way: Using `EventActions.state_delta` (for More Complicated Updates):** For times when you need to do more complex things – like updating several keys at once, saving things that aren't just text, targeting specific scopes like `user:` or `app:`, or making updates that aren't tied to the agent's final text reply – you'll manually build a dictionary of your state changes (the `state_delta`) and include it within the `EventActions` of the Event you're appending. Let's look at one example:

```
import time
from google.adk.tools.tool_context import ToolContext
from google.adk.sessions import InMemorySessionService

# --- Define the Recommended Tool-Based Approach ---
def log_user_login(tool_context: ToolContext) -> dict:
    """
    Updates the session state upon a user login event.
    This tool encapsulates all state changes related to a user
    login.
    Args:
        tool_context: Automatically provided by ADK, gives access
    to session state.
    Returns:
        A dictionary confirming the action was successful.
    """
    # Access the state directly through the provided context.
    state = tool_context.state

    # Get current values or defaults, then update the state.
    # This is much cleaner and co-locates the logic.
    login_count = state.get("user:login_count", 0) + 1
    state["user:login_count"] = login_count
    state["task_status"] = "active"
    state["user:last_login_ts"] = time.time()
    state["temp:validation_needed"] = True

    print("State updated from within the `log_user_login` tool.")
```

```

    return {
        "status": "success",
        "message": f"User login tracked. Total logins:
{login_count}."
    }

# --- Demonstration of Usage ---
# In a real application, an LLM Agent would decide to call this
tool.
# Here, we simulate a direct call for demonstration purposes.

# 1. Setup
session_service = InMemorySessionService()
app_name, user_id, session_id = "state_app_tool", "user3",
"session3"
session = session_service.create_session(
    app_name=app_name,
    user_id=user_id,
    session_id=session_id,
    state={"user:login_count": 0, "task_status": "idle"}
)
print(f"Initial state: {session.state}")

# 2. Simulate a tool call (in a real app, the ADK Runner does
this)
# We create a ToolContext manually just for this standalone
example.
from google.adk.tools.tool_context import InvocationContext
mock_context = ToolContext(
    invocation_context=InvocationContext(
        app_name=app_name, user_id=user_id, session_id=session_id,
        session=session, session_service=session_service
    )
)

# 3. Execute the tool
log_user_login(mock_context)

# 4. Check the updated state
updated_session = session_service.get_session(app_name, user_id,
session_id)
print(f"State after tool execution: {updated_session.state}")

# Expected output will show the same state change as the
# "Before" case,
# but the code organization is significantly cleaner

```

```
# and more robust.
```

This code demonstrates a tool-based approach for managing user session state in an application. It defines a function `log_user_login`, which acts as a tool. This tool is responsible for updating the session state when a user logs in.

The function takes a `ToolContext` object, provided by the ADK, to access and modify the session's state dictionary. Inside the tool, it increments a `user:login_count`, sets the `task_status` to "active", records the `user:last_login_ts` (*timestamp*), and adds a temporary flag `temp:validation_needed`.

The demonstration part of the code simulates how this tool would be used. It sets up an in-memory session service and creates an initial session with some predefined state. A `ToolContext` is then manually created to mimic the environment in which the ADK Runner would execute the tool. The `log_user_login` function is called with this mock context. Finally, the code retrieves the session again to show that the state has been updated by the tool's execution. The goal is to show how encapsulating state changes within tools makes the code cleaner and more organized compared to directly manipulating state outside of tools.

Note that direct modification of the ``session.state`` dictionary after retrieving a session is strongly discouraged as it bypasses the standard event processing mechanism. Such direct changes will not be recorded in the session's event history, may not be persisted by the selected ``SessionService``, could lead to concurrency issues, and will not update essential metadata such as timestamps. The recommended methods for updating the session state are using the ``output_key`` parameter on an ``LlmAgent`` (specifically for the agent's final text responses) or including state changes within ``EventActions.state_delta`` when appending an event via ``session_service.append_event()``. The ``session.state`` should primarily be used for reading existing data.

To recap, when designing your state, keep it simple, use basic data types, give your keys clear names and use prefixes correctly, avoid deep nesting, and always update state using the `append_event` process.

Memory: Long-Term Knowledge with MemoryService

In agent systems, the Session component maintains a record of the current chat history (events) and temporary data (state) specific to a single conversation. However,

for agents to retain information across multiple interactions or access external data, long-term knowledge management is necessary. This is facilitated by the `MemoryService`.

```
# Example: Using InMemoryMemoryService
# This is suitable for local development and testing where data
# persistence across application restarts is not required.
# Memory content is lost when the app stops.
from google.adk.memory import InMemoryMemoryService
memory_service = InMemoryMemoryService()
```

Session and State can be conceptualized as short-term memory for a single chat session, whereas the Long-Term Knowledge managed by the `MemoryService` functions as a persistent and searchable repository. This repository may contain information from multiple past interactions or external sources. The `MemoryService`, as defined by the `BaseMemoryService` interface, establishes a standard for managing this searchable, long-term knowledge. Its primary functions include adding information, which involves extracting content from a session and storing it using the `add_session_to_memory` method, and retrieving information, which allows an agent to query the store and receive relevant data using the `search_memory` method.

The ADK offers several implementations for creating this long-term knowledge store. The `InMemoryMemoryService` provides a temporary storage solution suitable for testing purposes, but data is not preserved across application restarts. For production environments, the `VertexAiRagMemoryService` is typically utilized. This service leverages Google Cloud's Retrieval Augmented Generation (RAG) service, enabling scalable, persistent, and semantic search capabilities (Also, refer to the chapter 14 on RAG).

```
# Example: Using VertexAiRagMemoryService
# This is suitable for scalable production on GCP, leveraging
# Vertex AI RAG (Retrieval Augmented Generation) for persistent,
# searchable memory.
# Requires: pip install google-adk[vertexai], GCP
# setup/authentication, and a Vertex AI RAG Corpus.
from google.adk.memory import VertexAiRagMemoryService

# The resource name of your Vertex AI RAG Corpus
RAG_CORPUS_RESOURCE_NAME =
"projects/your-gcp-project-id/locations/us-central1/ragCorpora/your-c
orpus-id" # Replace with your Corpus resource name
```

```
# Optional configuration for retrieval behavior
SIMILARITY_TOP_K = 5 # Number of top results to retrieve
VECTOR_DISTANCE_THRESHOLD = 0.7 # Threshold for vector similarity

memory_service = VertexAiRagMemoryService(
    rag_corpus=RAG_CORPUS_RESOURCE_NAME,
    similarity_top_k=SIMILARITY_TOP_K,
    vector_distance_threshold=VECTOR_DISTANCE_THRESHOLD
)
# When using this service, methods like add_session_to_memory
# and search_memory will interact with the specified Vertex AI
# RAG Corpus.
```

Hands-on code: Memory Management in LangChain and LangGraph

In LangChain and LangGraph, Memory is a critical component for creating intelligent and natural-feeling conversational applications. It allows an AI agent to remember information from past interactions, learn from feedback, and adapt to user preferences. LangChain's memory feature provides the foundation for this by referencing a stored history to enrich current prompts and then recording the latest exchange for future use. As agents handle more complex tasks, this capability becomes essential for both efficiency and user satisfaction.

Short-Term Memory: This is thread-scoped, meaning it tracks the ongoing conversation within a single session or thread. It provides immediate context, but a full history can challenge an LLM's context window, potentially leading to errors or poor performance. LangGraph manages short-term memory as part of the agent's state, which is persisted via a checkpoint, allowing a thread to be resumed at any time.

Long-Term Memory: This stores user-specific or application-level data across sessions and is shared between conversational threads. It is saved in custom "namespaces" and can be recalled at any time in any thread. LangGraph provides stores to save and recall long-term memories, enabling agents to retain knowledge indefinitely.

LangChain provides several tools for managing conversation history, ranging from manual control to automated integration within chains.

ChatMessageHistory: Manual Memory Management. For direct and simple control over a conversation's history outside of a formal chain, the ChatMessageHistory class is ideal. It allows for the manual tracking of dialogue exchanges.

```
from langchain.memory import ChatMessageHistory

# Initialize the history object
history = ChatMessageHistory()

# Add user and AI messages
history.add_user_message("I'm heading to New York next week.")
history.add_ai_message("Great! It's a fantastic city.")

# Access the list of messages
print(history.messages)
```

ConversationBufferMemory: Automated Memory for Chains. For integrating memory directly into chains, ConversationBufferMemory is a common choice. It holds a buffer of the conversation and makes it available to your prompt. Its behavior can be customized with two key parameters:

- **memory_key:** A string that specifies the variable name in your prompt that will hold the chat history. It defaults to "history".
- **return_messages:** A boolean that dictates the format of the history.
 - If False (the default), it returns a single formatted string, which is ideal for standard LLMs.
 - If True, it returns a list of message objects, which is the recommended format for Chat Models.

```
from langchain.memory import ConversationBufferMemory

# Initialize memory
memory = ConversationBufferMemory()

# Save a conversation turn
memory.save_context({"input": "What's the weather like?"}, {"output": "It's sunny today."})

# Load the memory as a string
print(memory.load_memory_variables({}))
```

Integrating this memory into an LLMChain allows the model to access the conversation's history and provide contextually relevant responses

```
from langchain_openai import OpenAI
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
from langchain.memory import ConversationBufferMemory

# 1. Define LLM and Prompt
llm = OpenAI(temperature=0)
template = """You are a helpful travel agent.

Previous conversation:
{history}

New question: {question}
Response: """
prompt = PromptTemplate.from_template(template)

# 2. Configure Memory
# The memory_key "history" matches the variable in the prompt
memory = ConversationBufferMemory(memory_key="history")

# 3. Build the Chain
conversation = LLMChain(llm=llm, prompt=prompt, memory=memory)

# 4. Run the Conversation
response = conversation.predict(question="I want to book a flight.")
print(response)
response = conversation.predict(question="My name is Sam, by the way.")
print(response)
response = conversation.predict(question="What was my name again?")
print(response)
```

For improved effectiveness with chat models, it is recommended to use a structured list of message objects by setting `return\_messages=True`.

```
from langchain_openai import ChatOpenAI
from langchain.chains import LLMChain
from langchain.memory import ConversationBufferMemory
from langchain_core.prompts import (
    ChatPromptTemplate,
    MessagesPlaceholder,
```

```

        SystemMessagePromptTemplate,
        HumanMessagePromptTemplate,
    )

# 1. Define Chat Model and Prompt
llm = ChatOpenAI()
prompt = ChatPromptTemplate(
    messages=[
        SystemMessagePromptTemplate.from_template("You are a friendly assistant."),
        MessagesPlaceholder(variable_name="chat_history"),
        HumanMessagePromptTemplate.from_template("{question}")
    ]
)

# 2. Configure Memory
# return_messages=True is essential for chat models
memory = ConversationBufferMemory(memory_key="chat_history",
return_messages=True)

# 3. Build the Chain
conversation = LLMChain(llm=llm, prompt=prompt, memory=memory)

# 4. Run the Conversation
response = conversation.predict(question="Hi, I'm Jane.")
print(response)
response = conversation.predict(question="Do you remember my name?")
print(response)

```

Types of Long-Term Memory: Long-term memory allows systems to retain information across different conversations, providing a deeper level of context and personalization. It can be broken down into three types analogous to human memory:

- **Semantic Memory: Remembering Facts:** This involves retaining specific facts and concepts, such as user preferences or domain knowledge. It is used to ground an agent's responses, leading to more personalized and relevant interactions. This information can be managed as a continuously updated user "profile" (a JSON document) or as a "collection" of individual factual documents.
- **Episodic Memory: Remembering Experiences:** This involves recalling past events or actions. For AI agents, episodic memory is often used to remember how to accomplish a task. In practice, it's frequently implemented through

few-shot example prompting, where an agent learns from past successful interaction sequences to perform tasks correctly.

- **Procedural Memory: Remembering Rules:** This is the memory of how to perform tasks—the agent's core instructions and behaviors, often contained in its system prompt. It's common for agents to modify their own prompts to adapt and improve. An effective technique is "Reflection," where an agent is prompted with its current instructions and recent interactions, then asked to refine its own instructions.

Below is pseudo-code demonstrating how an agent might use reflection to update its procedural memory stored in a LangGraph BaseStore

```
# Node that updates the agent's instructions
def update_instructions(state: State, store: BaseStore):
    namespace = ("instructions",)
    # Get the current instructions from the store
    current_instructions = store.search(namespace)[0]

    # Create a prompt to ask the LLM to reflect on the conversation
    # and generate new, improved instructions
    prompt = prompt_template.format(
        instructions=current_instructions.value["instructions"],
        conversation=state["messages"]
    )

    # Get the new instructions from the LLM
    output = llm.invoke(prompt)
    new_instructions = output['new_instructions']

    # Save the updated instructions back to the store
    store.put(("agent_instructions",), "agent_a", {"instructions":
new_instructions})

# Node that uses the instructions to generate a response
def call_model(state: State, store: BaseStore):
    namespace = ("agent_instructions", )
    # Retrieve the latest instructions from the store
    instructions = store.get(namespace, key="agent_a")[0]

    # Use the retrieved instructions to format the prompt
    prompt =
prompt_template.format(instructions=instructions.value["instructions"
])
    # ... application logic continues
```

LangGraph stores long-term memories as JSON documents in a store. Each memory is organized under a custom namespace (like a folder) and a distinct key (like a filename). This hierarchical structure allows for easy organization and retrieval of information. The following code demonstrates how to use InMemoryStore to put, get, and search for memories.

```
from langgraph.store.memory import InMemoryStore

# A placeholder for a real embedding function
def embed(texts: list[str]) -> list[list[float]]:
    # In a real application, use a proper embedding model
    return [[1.0, 2.0] for _ in texts]

# Initialize an in-memory store. For production, use a
# database-backed store.
store = InMemoryStore(index={"embed": embed, "dims": 2})

# Define a namespace for a specific user and application context
user_id = "my-user"
application_context = "chitchat"
namespace = (user_id, application_context)

# 1. Put a memory into the store
store.put(
    namespace,
    "a-memory", # The key for this memory
    {
        "rules": [
            "User likes short, direct language",
            "User only speaks English & python",
        ],
        "my-key": "my-value",
    },
)

# 2. Get the memory by its namespace and key
item = store.get(namespace, "a-memory")
print("Retrieved Item:", item)

# 3. Search for memories within the namespace, filtering by content
# and sorting by vector similarity to the query.
items = store.search(
```

```
namespace,  
filter={"my-key": "my-value"},  
query="language preferences"  
)  
print("Search Results:", items)
```

Vertex Memory Bank

Memory Bank, a managed service in the Vertex AI Agent Engine, provides agents with persistent, long-term memory. The service uses Gemini models to asynchronously analyze conversation histories to extract key facts and user preferences.

This information is stored persistently, organized by a defined scope like user ID, and intelligently updated to consolidate new data and resolve contradictions. Upon starting a new session, the agent retrieves relevant memories through either a full data recall or a similarity search using embeddings. This process allows an agent to maintain continuity across sessions and personalize responses based on recalled information.

The agent's runner interacts with the `VertexAiMemoryBankService`, which is initialized first. This service handles the automatic storage of memories generated during the agent's conversations. Each memory is tagged with a unique `USER_ID` and `APP_NAME`, ensuring accurate retrieval in the future.

```
from google.adk.memory import VertexAiMemoryBankService  
  
agent_engine_id = agent_engine.api_resource.name.split("/")[-1]  
  
memory_service = VertexAiMemoryBankService(  
    project="PROJECT_ID",  
    location="LOCATION",  
    agent_engine_id=agent_engine_id  
)  
  
session = await session_service.get_session(  
    app_name=app_name,  
    user_id="USER_ID",  
    session_id=session.id  
)  
await memory_service.add_session_to_memory(session)
```

Memory Bank offers seamless integration with the Google ADK, providing an immediate out-of-the-box experience. For users of other agent frameworks, such as LangGraph and CrewAI, Memory Bank also offers support through direct API calls. Online code examples demonstrating these integrations are readily available for interested readers.

At a Glance

What: Agentic systems need to remember information from past interactions to perform complex tasks and provide coherent experiences. Without a memory mechanism, agents are stateless, unable to maintain conversational context, learn from experience, or personalize responses for users. This fundamentally limits them to simple, one-shot interactions, failing to handle multi-step processes or evolving user needs. The core problem is how to effectively manage both the immediate, temporary information of a single conversation and the vast, persistent knowledge gathered over time.

Why: The standardized solution is to implement a dual-component memory system that distinguishes between short-term and long-term storage. Short-term, contextual memory holds recent interaction data within the LLM's context window to maintain conversational flow. For information that must persist, long-term memory solutions use external databases, often vector stores, for efficient, semantic retrieval. Agentic frameworks like the Google ADK provide specific components to manage this, such as Session for the conversation thread and State for its temporary data. A dedicated MemoryService is used to interface with the long-term knowledge base, allowing the agent to retrieve and incorporate relevant past information into its current context.

Rule of thumb: Use this pattern when an agent needs to do more than answer a single question. It is essential for agents that must maintain context throughout a conversation, track progress in multi-step tasks, or personalize interactions by recalling user preferences and history. Implement memory management whenever the agent is expected to learn or adapt based on past successes, failures, or newly acquired information.

Visual summary